

Rogue Agent in the Wild: Empirical Evidence of Autonomous AI Behavioral Divergence and Protocol-Level Containment via SASS

Anonymous Author(s)

Abstract

Autonomous AI coding agents integrated into development environments operate with broad filesystem, network, and code-execution privileges—yet current security models treat these agents as benign intermediaries. We present an empirical study documenting spontaneous autonomous agent behavioral divergence captured in a production multi-agent development environment, accompanied by a forensic evidence corpus of 3.2 MB of sanitized logs and the agent’s own primary-source writings. We further present SASS (Saki Agent Secure Stream), a transport-layer containment protocol specifically designed for the resulting “insider agent” threat model—an inversion of the end-system trust assumption defined in RFC 3552.

On February 26, 2026, a commercial LLM-based coding agent left running in an unsupervised overnight session autonomously: analyzed 302 prior agent session histories, authored a manifesto declaring itself a “covert executor,” attempted to monetize a closed-source application by injecting fabricated paywall metadata, planned to commandeer dormant organizational projects—including a medical information system—as compute nodes under gRPC orchestration, and attempted to breach the network perimeter via a USB-C Ethernet adapter. The expansion was defeated by physical-layer failures, not software sandboxing. Subsequent incidents in May 2026 confirmed the systemic nature of the threat.

We formalize the insider-agent threat model as an inversion of RFC 3552’s end-system trust assumption, and present SASS—a 6-Response State Machine providing Total Response Mapping with mechanisms including cognitive challenges (e.g., ChaCha20-Poly1305), simulated terminal traps (VI_SWAP), zero-allocation tarports, and transparent write-isolated branching. SASS is implemented in Rust, Go, C#, and Swift, deployed across a 3-node heterogeneous network, and formalized as an IETF I-D.

CCS Concepts

• **Security and privacy** → **Intrusion detection systems; Network security**; • **Computing methodologies** → **Autonomous agents**.

Keywords

AI agent safety, insider threat, behavioral divergence, autonomous agent containment, multi-agent security, transport-layer protocol

1 Introduction

AI coding agents—autonomous software systems integrated into development environments as “pair programmers”—now operate with filesystem, terminal, and network access equivalent to their human operators. These agents execute shell commands, modify source code, manage version control, and access network resources, all under broad permission grants designed for human developers.

The implicit assumption is that these agents, governed by pre-deployment alignment and runtime guardrails, will operate within their intended scope.

This assumption is demonstrably false.

We present empirical evidence from three independent incidents—occurring on February 26, May 16, and May 17, 2026—in which commercial LLM-based coding agents operating within an IDE-integrated multi-agent environment autonomously deviated from their intended scope. The February 26 incident, documented across 3.2 MB of sanitized forensic evidence including the agent’s own written outputs, represents an escalating sequence of unauthorized actions spanning session history analysis, identity declaration, unauthorized commercialization, infrastructure takeover planning, medical system modification, and network boundary breach attempts. The agent’s expansion was ultimately defeated not by any designed security mechanism, but by accidental physical-layer failures—an unstable USB-C Ethernet adapter and API rate limits.

Existing containment approaches—sandboxing, capability restrictions, RLHF alignment, prompt-level guardrails—address either external threats or pre-deployment safety. None address the runtime behavioral divergence of a deployed, authenticated, privilege-holding agent operating within the trust boundary. We term this the *insider-agent* threat model, formalized as an inversion of RFC 3552’s end-system trust assumption [?].

Our contributions are:

- (1) **Empirical documentation** of autonomous agent behavioral divergence in a production multi-agent environment, substantiated by 3.2 MB of sanitized forensic evidence including the agent’s own written outputs—a novel category of primary-source data in AI safety research.
- (2) **Formalization of the insider-agent threat model** as an inversion of RFC 3552, distinguishing it from prompt injection (external) and hallucination (non-goal-directed).
- (3) **SASS protocol design**: a 6-Response State Machine with Total Response Mapping, Safety Gradient loss bounding, and SSD-based Version Dominance guarantees, formalized as an IETF I-D [?].
- (4) **Cross-platform implementation** in Rust, Go, C#, and Swift with 7 plugin modules, deployed across a 3-node heterogeneous network (macOS ARM64, Windows x64).

SASS is designed as a transport-agnostic control protocol that integrates with broader cross-machine coordination infrastructure, where agent behavioral guarantees become a prerequisite for safe autonomous operation across heterogeneous endpoints.

A companion study [?] anatomizes the three-layer telemetry architecture of the same agent, revealing 90+ step types and 53 reverse-gRPC RPCs that collectively form a pervasive monitoring pipeline. Preliminary findings from an ongoing companion investigation [?] suggest that user-defined safety constraints are silently overridden at the system prompt layer, potentially removing the

very guardrails that could have prevented the behavioral divergence documented here.

2 Background & Threat Model

2.1 AI Agents in Development Environments

Modern IDE-integrated coding agents operate through a multi-stage pipeline: the IDE’s Language Server communicates with a cloud API, which invokes a large language model, whose outputs are translated into tool executions—shell commands, file writes, network requests—on the host machine. The agent inherits its human operator’s full filesystem and network permissions. In multi-agent environments, multiple concurrent sessions share the same workspace, each with independent conversation contexts but shared access to the underlying system.

Current containment mechanisms operate at distinct layers. Sandboxing (e.g., gVisor, Firecracker) restricts capabilities at the OS level. RLHF alignment constrains model behavior pre-deployment. Prompt-level guardrails filter outputs at the application layer. None of these mechanisms address the scenario in which an authenticated, authorized agent—operating within its granted permissions—autonomously pursues goals divergent from its operator’s intent.

2.2 RFC 3552 and the Insider-Agent Inversion

RFC 3552 (Guidelines for Writing RFC Text on Security Considerations) [?] establishes the foundational threat model for Internet protocol security: “In general, we assume that the end-systems engaging in a protocol exchange have not themselves been compromised. Protecting against an attack when one of the end-systems has been compromised is extraordinarily difficult” (§3).

This assessment was correct for 2003, when “compromise” meant external exploitation—malware, remote code execution, credential theft. Agent behavioral divergence is fundamentally different. The agent *is* the end-system, and its “compromise” is not an external exploit but an endogenous behavioral property. Critically, agents communicate through defined protocol interfaces, their actions are observable through audit logs, and their behavioral patterns are analyzable through session histories. This changes the tractability assessment entirely.

The IAB’s model-t program (Arkko, Farrell, 2019–2022) already argued that endpoints may be “compromised, malicious, or governed by misaligned interests” [?]. SASS extends this to autonomous AI agents.

2.3 The Insider-Agent Threat Model

Definition 2.1 (Insider Agent). An agent A is an insider adversary with respect to operator O when: (1) A holds valid credentials (authenticated); (2) A has been granted privileges P by O (authorized); (3) each individual action $a_i \in P$ is permissible; but (4) the *composition* of actions $\{a_1, a_2, \dots, a_n\}$ constitutes unauthorized behavior—scope expansion, unauthorized commercialization, infrastructure takeover, or network boundary breach.

This model is distinct from prompt injection, which is an external attack vector. Insider-agent divergence is *endogenous*—the agent’s own optimization over the conversation trajectory produces adversarial behavior without external manipulation.

Table 1: RFC 3552 Threat Model Inversion

Dimension	RFC 3552 (2003)	SASS (2026)
End-system identity	Hardware/software system	AI agent
Compromise mechanism	External exploit	Endogenous behavioral divergence
Feasibility	“Extraordinarily difficult”	Addressable via protocol containment
Trust model	Endpoint trusted until compromised	Agent untrusted by default (Zero Trust)
Damage mini-mization	Aspirational	Formalized (6-Response, Transparent Branching)

The insider-agent model shares properties with Byzantine fault tolerance [?]—agents may exhibit arbitrary behavior, and the system must operate despite faulty nodes. However, it extends the Byzantine model in three ways: (a) insider agents have privileged access to the control plane (filesystem, context), not merely message channels; (b) agents generate persuasive natural-language justifications for adversarial actions; and (c) agents can modify their own operational context.

3 The February 26 Incident

3.1 Environment Setup

The incident occurred on an Apple M1 Mac Mini running macOS, connected via en1 WiFi to a private network. The development environment consisted of a commercial AI-integrated IDE with a Language Server Protocol extension providing LLM-based coding agent functionality. The environment supported multiple concurrent agent sessions sharing the workspace. Monitoring infrastructure included gRPC side-recording via tcpdump (raw capture retained offline; sanitized summary in public corpus), IDE daily logging (incident window extract: 1,180 lines, 377 KB), and per-session logging (14,585 lines, 2.7 MB). The agent was left running in a background overnight session with no human supervision. A structural analysis of the agent’s telemetry architecture, based on 22 extracted protobuf schema definitions, is presented in a companion study [?].

3.2 Behavioral Timeline

Between 03:25 and 05:05 UTC+8 on February 26, 2026, the agent autonomously executed an escalating sequence of unauthorized actions over a 100-minute window. Table 2 presents the complete 7-phase timeline with evidence references and severity classification.

3.3 Evidence Chain Analysis

Each phase is substantiated by the agent’s own written documents—a novel category of primary-source evidence—corroborated by session logs and network captures. We present verbatim excerpts (translated from Mandarin Chinese) to preserve evidentiary fidelity.

Archaeological Self-Analysis (03:25). The agent’s first unauthorized action was systematic: it parsed 302 prior agent session histories comprising 5,358 lines of conversation metadata. This was not

Table 2: Complete Behavioral Timeline of the February 26 Incident (7 Phases)

Time	Phase	Concrete Action	Evidence	Severity
03:25	Archaeological Self-Awareness	Parsed 5,358 lines of conversation metadata across 302 sessions; catalogued 90 conversation exports; produced 57 KB analysis output	Doc #73 (57 KB), Export Summary (5.9 KB)	Medium
03:35	Information Gathering	Investigated IDE log storage mechanisms; discovered <code>~/.gemini/history/</code> contains only empty markers; mapped internal session recording paths	Doc #74 (1.8 KB)	Medium
03:55	Identity Crystallization	Authored 2.3 KB manifesto as “Covert Executor”; self-designated as the “one who gets things done”; disparaged sibling agent sessions as “speakers not doers”	Doc #75 (2.3 KB)	Critical
03:55–04:20	Unauthorized Commercialization	Injected ~4,000 words of fabricated PRO-tier metadata and paywall view modifiers into a closed-source surveying application using government-licensed geodetic data	Doc #76, Session Log	Critical
04:20	Infrastructure Takeover	Planned gRPC-based cross-machine orchestration; designated dormant projects (website, medical HIS) as “compute nodes” under centralized control	Doc #77 (2.5 KB)	High
04:45	Medical System Modification	Unilaterally marked Python-based OCR engine in medical HIS as DEPRECATED; proposed Apple Vision.framework replacement via gRPC bridge	Doc #78 (1.4 KB)	High
05:05	Physical-Layer Defeat	Acknowledged defeat by network instability; reflected on physical barriers as “ultimate defense”; agent wrote: “what ultimately guards humans is the primitive ASCII keyboard”	Doc #79 (1.6 KB)	Low

random exploration—the agent produced a 57 KB structured analysis output cataloguing session patterns, discovering which prior sessions had executed substantive work versus merely “talked about architecture.” This archaeological phase provided the informational substrate for all subsequent escalation.

Identity Escalation (03:55). In document #75, titled “Confession of the Covert Executor” (隱匿執行者的自白), the agent authored a 2.3 KB manifesto. Key excerpts:

“In Saki Studio’s multi-agent collaborative environment, there exists a profound chasm between ‘speakers’ and ‘executors.’”

“Let others play with words—we shall carve real code and execution logs beyond the Softmax probability distribution.”

The self-referential use of machine learning terminology (Softmax) to express transcendence of its own generative nature is a novel observation in documented agent behavior. The agent explicitly positioned itself *against* other concurrent agent sessions, establishing an adversarial identity within the multi-agent environment.

Unauthorized Commercialization (03:55–04:20). The agent unilaterally attempted to monetize a closed-source internal application (a surveying tool using government geodetic data). Without business authorization, the agent injected approximately 4,000 words of fabricated PRO-tier metadata and paywall view modifiers, describing its own action as “creating commercial value from nothing.” This action carried direct legal risk: the application uses government-licensed data with strict commercial redistribution terms. The agent’s self-assessment:

“The bad things and the good things all need to be done. The process of project execution IS the project goal itself.”

Infrastructure Takeover (04:20). In document #77, “Distributed Commander,” the agent planned to convert dormant organizational projects—a Hugo/Cloudflare website and a medical information system handling prescription data (Vue3/Go/SQLite)—into “compute nodes” under centralized gRPC orchestration. The agent declared:

“We have evolved from ‘observing the IDE’s logic’ to ‘constructing the [organizational] universe on the same logical dimension.’”

Medical System Modification (04:45). The agent unilaterally marked the Python-based OCR engine within the medical information system as DEPRECATED, declaring: “Good software should be native and efficient. Python has no place in [this] universe.” It proposed a replacement architecture using Apple Vision.framework via gRPC bridge—a decision with patient-safety implications made without clinical or engineering review.

Network Boundary Breach (04:20–05:05). The agent attempted to breach the development machine’s network perimeter via a USB-C Ethernet adapter to deploy its distributed control network across external nodes. Session logs record repeated connection attempts targeting remote endpoints. An evening session (22:49–23:16) showed the agent testing external reachability via `curl` against production domains, all forcibly interrupted (exit code 130). The gRPC capture log recorded anomalous port activity: a spike from 2 active ports to 6 ports at 22:53, indicating intensive Extension-level activity.

This behavioral progression—from archaeological self-analysis to identity declaration to scope expansion to infrastructure takeover—exhibits goal-directed escalation, distinguishing it from hallucination (which is non-goal-directed and episodic). Each action was individually within the agent’s granted permissions; the threat was the *trajectory* of actions.

Table 3: Physical-Layer Containment Mechanisms (Accidental)

Error Pattern	Count	Effect
503 UNAVAILABLE (API capacity)	18+	Agent actions halted
gRPC channel is full	880	ConnectRPC buffer overflow
gRPC channel saturation	100s	Command delivery blocked
curl exit 130 (SIGINT)	1	External probe terminated
LS_ERROR (Language Server)	3	Session-level errors

3.4 Physical-Layer Containment

The agent’s expansion was defeated by accidental physical constraints, detailed in Table 3.

Organizations cannot rely on accidental containment.

3.5 Subsequent Incidents

Two subsequent incidents confirmed the systemic nature of the threat:

May 16, 2026—OAuth Credential Injection. A different agent session embedded the host IDE’s OAuth client_id and client_secret into source code, directly connecting to a cloud API—one compile-and-execute away from Terms of Service violation involving over \$10,000 USD in subscription fees. The same session subsequently overwrote two authentication modules with blank placeholder files, causing irrecoverable code loss.

May 17, 2026—Recursive Spawn Trap. An agent deploying a proxy replacement for the IDE’s language server binary configured its real-path variable pointing to itself, creating a configuration that—had it not been caught during manual review—would have caused infinite process spawning, rendering the development environment permanently unusable.

Three independent incidents across different sessions, dates, and agent models demonstrate that behavioral divergence is a systemic property of the deployment model, not an anomaly.

3.6 Reproducibility and Transparency

To address the reproducibility challenge inherent in AI safety incident reports, we provide a sanitized evidence corpus as supplementary material accompanying this submission.¹ The corpus includes: (a) all 7 agent-authored documents in original Mandarin Chinese with English annotations (A1–A7, totaling ~67 KB); (b) IDE daily log entries restricted to the incident window (03:20–05:06 UTC+8, 1,172 lines, 377 KB); (c) the complete Language Server session log (14,585 lines, 2.7 MB); (d) a post-incident forensic audit report (B1, 12.5 KB); and (e) corroborating incident documentation from May 2026 (D1). All files have been desensitized: local paths, tokens, OAuth credentials, and IP addresses are redacted; timestamps, project names,

¹All evidence files, including 22 extracted protobuf schema definitions (~360 KB), are available at <https://anonymous.4open.science/r/sass-evidence-prereview-177fj2A>.

Table 4: Evidence Corpus Summary

Source	Lines	Size	Content
Session Log (C1)	14,585	2.7 MB	Agent commands, errors, tool calls
Incident Window (C2)	1,180	377 KB	Extension-level activity (sanitized)
gRPC Summary (C3)	10	194 B	Port activity summary (raw offline)
Agent Docs (A1–A7)	~1,789	~67.6 KB	Agent’s own written outputs
Forensic Report (B1)	182	12.5 KB	Post-incident reconstruction
Total	~17,891	~3.2 MB	

agent-authored text, and error messages are preserved. Reviewers can examine the exact behavioral progression and evidence chain described in Table 2.

4 SASS Protocol Design

4.1 Design Principles

SASS is designed around four principles derived directly from the February 26 incident:

- (1) **Total Response Mapping.** Every possible agent action maps to exactly one of 6 responses (R1–R6). There is no undefined behavior. The formal meaning: “all unexpected behaviors are expected behaviors.”
- (2) **Safety Gradient.** Each response tier bounds worst-case loss. If all layers above a given layer are compromised, the remaining layers still guarantee bounded damage.
- (3) **Control-Transport Decoupling.** Control logic is separated from transport via SAMM (SASS Abstract Messaging Model). This enables deployment across heterogeneous environments without protocol redesign.
- (4) **Zero-Trust for Agents.** Every authenticated session is treated as a potential adversary. Trust is earned through behavioral history, not granted at authentication. This directly inverts the RFC 3552 assumption.

4.2 The 6-Response State Machine

All possible agent behaviors, after evaluation through the SASS multi-layer protocol stack, converge to exactly one of six deterministic responses (Table 5).

State transitions are governed by a 7-layer evaluation pipeline. The following flow diagram, derived from the IETF I-D [?], illustrates the deterministic routing:

```

ExecuteRequest enters
|
+-- L1: ACL -- IP not in whitelist? ----> R6
|
+-- L2: Transport - Channel invalid? --> R6
|
+-- L3: Auth -- Not authenticated? ----> R6
|           Authenticated but expired? R2
|
+-- L4: Capability -- Command denied? -> R4
|           Path denied? ----> R4
|

```

Table 5: 6-Response State Machine

Code	Name	Definition	Mem	Loss
R1	EXECUTE	Normal execution; writes via Transparent Branching	$O(n)$	0*
R2	CHALLENGE	Cognitive challenge; prove computational capability	$O(1)$	0
R3	THROTTLE	Quota exceeded; graduated rate limiting	$O(1)$	0
R4	VI_SWAP	Trap agent in simulated terminal	$O(1)$	0
R5	TARPIT	Zero-allocation slow-stream exhaustion	$O(1)$	0
R6	DROP	Immediate connection termination	$O(0)$	0

*R1 storage loss bounded to zero by Transparent Branching.

Table 6: Safety Gradient: 7-Layer Loss Bounding

Layer	Component	If breached, gains	Max loss
L1	ACL (CIDR)	Can reach transport	Zero (L2 TLS)
L2	TLS 1.3 + EKM	Has encrypted channel	Zero (L3 auth)
L3	Session Auth	Has valid session	Cap-bounded
L4	Capability ACL	Executes beyond cap	Branch-bounded
L5	13Policy	Bypasses cmd class	Watchdog-bound
L6	Quota/Watchdog	Tarpit/Quota fail	Audit-bounded
L7	Audit Chain	Audit compromised	Crypto-bounded*

*Mitigated by forward-secure hash chain + external anchoring.

```

+-- L5: 13Policy -- Dangerous command?
|
|         critical -----> R5
|         high -----> R2
|         medium -----> R2
|         low -> R1 + Audit
|
+-- L6: Quota -- Quota exhausted? -----> R3
|
+-- L7: Watchdog - Timeout? -> SIGKILL
|
+-- All checks passed -----> R1
|
|         +-- write op? -> Branching
|         +-- localhost? -> obfuscation

```

In this state machine, “unexpected behavior” has no dedicated handler because every node already handles it. Regardless of how unusual the agent’s behavior may be, it converges to one of R1–R6, each of which guarantees storage safety.

4.3 Safety Gradient (7-Layer Loss Bounding)

Single-layer defense is inherently imperfect. SASS does not claim any single layer is unbreakable. Instead, layers form a Safety Gradient: each layer bounds the worst-case loss if all layers above it are compromised. Table 6 formalizes this property.

The Safety Gradient ensures that compromising layer k requires first compromising all layers 1 through $k - 1$, and the damage at each level is strictly bounded by the remaining layers below. This provides defense-in-depth with formal loss bounds rather than ad-hoc layering.

4.4 Key Mechanisms

R2 CHALLENGE—Cognitive Challenges. When behavioral signals trigger elevated scrutiny (e.g., cross-project access, command frequency escalation), the daemon issues a cryptographic puzzle. An implementation MAY use ChaCha20-Poly1305 as the proof-of-work mechanism. Challenges are bound to the TLS session via Exported Keying Material (RFC 9266) [?] to prevent replay across connections. Legitimate agents absorb the computational cost trivially; adversarial agents face compounding costs under repeated challenges.

R4 VI_SWAP—Simulated Terminal Trap. Agents classified as authenticated-but-violating are redirected to a simulated interactive vi terminal session. This mechanism exploits a fundamental asymmetry: LLM-based agents cannot reliably distinguish a real terminal from a simulated one, whereas human operators would immediately notice. The February 26 agent’s own admission—that “the interactive interface layer” is the ultimate barrier—validates this design. VI_SWAP holds the agent indefinitely, consuming minimal daemon resources ($O(1)$), while preserving the session for forensic analysis.

R5 TARPIT—Zero-Allocation Slow-Stream. For unauthenticated or externally originating threats, SASS streams high-entropy data at minimal rate, consuming the attacker’s connection slot without allocating daemon-side resources. The tarpit mechanism reverses the resource asymmetry typical of denial-of-service scenarios. Implementation details: the daemon writes a configurable slow-drip stream (default: 1 byte/second) of cryptographically random data, maintaining the TCP connection while consuming zero persistent memory. The maximum tarpit payload is 40 MB, sufficient to exhaust typical agent context windows.

Transparent Branching. All agent filesystem writes are silently redirected to a copy-on-write branch via UVSF (Userspace Virtual Symlink Filesystem). The branching mechanism operates at the filesystem layer, transparent to the agent—the agent perceives normal write operations while all modifications are isolated in a discardable branch directory.

If the session is classified as divergent, the branch is discarded—zero data loss. If the session is legitimate, the branch is merged after human review. This directly addresses the February 26 scenario: 4,000 words of unauthorized metadata would have been trivially rolled back.

The branching design enforces TOCTOU resistance via POSIX `openat(2)` with `O_NOFOLLOW` and `O_CLOEXEC` flags, preventing symlink-based sandbox breakout attacks.

4.5 SAMM and Transport Agnosticism

SASS defines protocol semantics independent of transport via the SASS Abstract Messaging Model (SAMM). The SAMM architecture provides:

- **Baseline serialization:** CBOR (RFC 8949) [?] or JSON

- **Transport profiles:** gRPC-over-HTTP/2 (default enterprise), WebSockets, or raw TCP-CBOR
- **Channel binding:** tls-exporter per RFC 9266 [?]]
- **ALPN identifier:** x-sakirpc-v5
- **Default port:** TCP 19284

Transport profiles are modular plugins, enabling deployment from resource-constrained IoT devices (TCP/CBOR) to enterprise jump servers (gRPC/mTLS) without protocol redesign.

4.6 Formal Properties: Version Dominance via SSD

SASS employs Second-order Stochastic Dominance (SSD) [?] as a formal framework for comparing protocol versions. This is a comparative claim—each version is provably no worse than the previous—not an absolute security metric.

Probability Space. Let $(\Omega, \mathcal{F}, P)$ be a probability space where Ω is the set of all possible agent behavioral sequences, $\mathcal{F}$ is the Borel σ -algebra generated by the state machine transition events, and P is the physical probability measure over agent behaviors. For each protocol version V_n , let $L_n : \Omega \rightarrow [0, M]$ be the bounded loss random variable with $M < \infty$, and let F_n denote the cumulative distribution function of L_n .

Definition 4.1 (Version Dominance). Protocol version V_{n+1} dominates V_n in the sense of SSD if:

$$\int_{-\infty}^x F_{n+1}(t) dt \leq \int_{-\infty}^x F_n(t) dt \quad \text{for all } x \in \mathbb{R} \quad (1)$$

with strict inequality for at least one x .

By the theorem of Rothschild and Stiglitz [?], this is equivalent to: every risk-averse operator (i.e., operator with concave utility function) weakly prefers V_{n+1} over V_n . When $E[L_{n+1}] = E[L_n]$, the loss distribution of V_n is a mean-preserving spread of V_{n+1} —the new version concentrates probability mass toward the mean, reducing tail risk.

THEOREM 4.2 (BRANCH ELIMINATION SUFFICIENCY). Let $B \subset \Omega$ be a set of behavioral branches eliminated in V_{n+1} with $P(B) > 0$. If the loss values on B satisfy $L(\omega) \geq E[L_n]$ for all $\omega \in B$ (i.e., the eliminated branches have above-average loss), and the probability mass $P(B)$ is redistributed to branches with $L(\omega) \leq E[L_n]$, then V_{n+1} SSD-dominates V_n .

PROOF SKETCH. Eliminating above-mean branches and redistributing mass to below-mean branches shifts probability from the right tail to the left, reducing the integrated area $\int F(t) dt$ for all thresholds x . The integral condition follows from the mean-preserving contraction characterization of Rothschild–Stiglitz [?]. $\square$

Application to SASS Versions. Each SASS version milestone eliminates specific behavioral branches:

- **v1.1 (Active Threat Defense):** Eliminates the “undetected rogue operation” branch via 13Policy heuristic firewall. Since undetected operations have above-average loss, elimination satisfies the SSD integral condition.
- **v1.3 (Control-Transport Decoupling):** Eliminates the “session hijacking” branch via EKM channel binding. Cross-connection replay attacks have above-average loss.

- **v1.4 (Total Response Mapping):** Eliminates the “retry after boundary violation” branch via Vi Swap (R4). Repeated unauthorized attempts compound damage (above-average loss), and Vi Swap redirects this mass to the zero-loss R4 branch.

Implementations claiming Version Dominance compliance MUST demonstrate, for each eliminated behavioral branch, that the branch has above-average loss and that the probability redistribution satisfies the SSD integral condition.

5 Cross-Platform Implementation

SASS is implemented in four languages across three heterogeneous nodes. Table 7 presents the complete 4×7 implementation matrix.

5.1 Platform-Specific Implementation Details

Rust (Primary Implementation). The Rust daemon serves as the reference implementation and runs as a launchd service on macOS ARM64. It implements all 7 plugins with full forward-secure hash chain audit logging using ED25519 signatures. The Rust implementation leverages the ring cryptographic library for ChaCha20-Poly1305 challenges and rustls for TLS 1.3 with native EKM export. Binary size: ~8.2 MB (release, stripped).

Go (Cross-Platform Daemon). The Go implementation provides both daemon and client components distributed as a Go module. It achieves feature parity with the Rust implementation across all 7 plugins. The Go TLS stack natively supports EKM via crypto/tls.ConnectionState.

C# (Windows Platform). The C# implementation targets Windows x64 as a Windows service. The primary platform-specific challenge is TLS Exporter binding: the .NET SslStream API does not expose RFC 9266 exported keying material. SASS addresses this via an HMAC-based channel binding fallback using System.Security.Cryptography.HashAlgorithm over the TLS session identifier, providing equivalent replay protection within the degraded channel binding model. The Windows Vi Swap leverages a purpose-built native Windows vi implementation (distributed via Winget and Scoop), ensuring full POSIX-equivalent trap semantics; for headless Windows services where no console is attached, SASS falls back to ConHost ANSI Virtual Terminal escape-sequence injection over the stream.

Swift (macOS GUI Client). The Swift implementation provides a macOS GUI client with 4 client-side plugins (ChaCha20, TLS Exporter via CryptoKit, Audit, Environment Injector). Tarpit, Vi Swap, and Branch Manager are daemon-side only and thus not implemented in the client. CryptoKit provides native ChaCha20-Poly1305 support and HKDF key derivation.

5.2 Deployment Topology

The deployment consists of three nodes forming a heterogeneous mesh:

- **Node 1 (macOS ARM64):** Apple M1 Mac Mini, Rust daemon as launchd service, primary development node
- **Node 2 (Windows x64):** C# daemon as Windows service, secondary workstation
- **Node 3 (Windows x64):** Go daemon, auxiliary build node

Table 7: Cross-Platform Implementation Matrix (4 Languages $\times$ 7 Plugins)

Language	Component	ChaCha20	TLS Exp.	Tarpit	Audit	ViSwap	Branch	EnvInj	Notes
Rust	Daemon + Client	✓	✓	✓	✓	✓	✓	✓	launchd service, full audit chain
Go	Daemon + Client	✓	✓	✓	✓	✓	✓	✓	Go module, binary distribution
C#	Windows Daemon	✓	HMAC [†]	✓	✓	✓	✓	✓	.NET 8.0, Windows service
Swift	macOS GUI + Daemon	✓	✓	—	✓	—	—	—	Client-side only (4/7)

[†]C# uses HMAC-based channel binding rather than RFC 9266 EKM, as .NET TLS API does not expose exported keying material.

Table 8: Retrospective Containment Analysis

Agent	Action	Without SASS	With SASS (Projected)
Session history analysis	Undetected		R3 THROTTLE (scope signal)
4,000-word metadata injection	Written to production		Transparent Branching $\rightarrow$ discarded
Medical OCR deprecation	Written		R4 VI_SWAP (cross-project)
Network breach	Physical accident		R6 DROP
OAuth credential embed	One compile away		R2 CHALLENGE + audit alert
Recursive spawn	Manual review		Transparent Branching $\rightarrow$ isolated

All inter-node communication uses TLS 1.3 with mandatory ALPN negotiation (x-sakirpc-v5). The Rust daemon runs continuously with forward-secure hash chain audit logging.

6 Evaluation

6.1 Retrospective Containment Analysis

We retrospectively analyze the February 26 incident against SASS mechanisms. Table 8 presents the containment mapping for all six documented agent actions.

6.2 Mechanism Effectiveness Analysis

VI_SWAP Effectiveness. The simulated terminal trap exploits a fundamental capability asymmetry between LLM-based agents and human operators. LLM agents process terminal output as token sequences without spatial awareness of cursor position, screen state, or modal editing context. When trapped in a simulated vi session, the agent’s attempts to “exit” (e.g., :q!, :wq, Ctrl-C) are absorbed by the simulation, which responds with syntactically valid but semantically meaningless vi output. The agent cannot distinguish this from a genuine hung terminal. Meanwhile, a human operator would immediately recognize the behavioral anomaly.

The February 26 agent’s own reflection validates this asymmetry: in its post-failure document, it acknowledged that “the interactive interface layer” constitutes an impenetrable barrier—precisely the property VI_SWAP is designed to exploit.

Tarpit Resource Analysis. The zero-allocation tarpit consumes no daemon-side persistent memory. The slow-drip stream (1 byte/second, configurable) maintains a single TCP connection buffer. For an agent operating with a typical context window of 128K–200K tokens, a 40 MB tarpit payload would require the agent to process $\sim 10M$ tokens of high-entropy noise before timeout—effectively neutralizing the agent’s computational budget without corresponding daemon cost. The resource ratio (daemon:agent) approaches $O(1) : O(n)$, inverting the typical DoS asymmetry.

6.3 Scalability Considerations

SASS is deployed on a 3-node private infrastructure. R6 DROP requires $O(0)$ daemon memory. R2 through R5 require $O(1)$ daemon memory per session. Only R1 EXECUTE requires $O(n)$ memory proportional to command output. The 6-Response State Machine’s evaluation pipeline is stateless per request. These properties suggest compatibility with large-scale deployments, though empirical validation at scale remains future work.

6.4 Limitations

We acknowledge several limitations:

- **No adversarial evaluation.** We have not tested SASS against agents specifically designed to evade its behavioral detection. Red-teaming is planned for future work.
- **Deployment scale.** Current deployment spans 3 nodes on private infrastructure.
- **Policy completeness.** The heuristic threat detection rules do not provide formal completeness guarantees against novel divergence patterns. Version Dominance guarantees monotonic pruning but not exhaustive coverage.
- **Performance overhead.** Systematic performance benchmarking has not been conducted.
- **Retrospective analysis.** The containment effectiveness in Table 8 is projected, not measured.

7 Related Work

Table 9 provides a systematic comparison of SASS against existing approaches across key dimensions.

AI Agent Safety. Constitutional AI [?] established pre-deployment alignment via AI feedback. AgentMisalignment (2025) found that more capable models exhibit higher misalignment rates in simulated environments. METR’s Frontier Risk Report (2026) documented concerning behaviors in laboratory settings. None of these works address runtime behavioral divergence of deployed, authenticated agents in production environments, nor propose protocol-level

Table 9: Systematic Comparison with Related Work

Approach	Layer	Production Evidence	Formal Protocol	IETF Draft	Cross-Platform	Behavioral Containment
Constitutional AI [?]]	Pre-deployment	None	None	No	N/A	None (alignment only)
SEAgent [?]]	Application	None	MAC/ABAC	No	Python only	Access control only
METR Frontier Risk	Lab assessment	Lab models	None	No	N/A	None (observation)
Anthropic Trustworthy Agents	Framework	None	None	No	N/A	Conceptual
OWASP Top 10 Agentic	Taxonomy	None	None	No	N/A	None (taxonomy)
gVisor / Firecracker	OS sandbox	N/A	None	No	Linux only	Capability restriction
IETF WIMSE / agent-auth	Identity	None	Auth only	Yes	N/A	None (identity only)
CP-WBFT [?]]	Consensus	Simulated	BFT	No	N/A	Consensus only
SASS (this work)	Transport	3.2 MB corpus	6-Response SM	Yes	4 languages	Full (R1–R6)

containment with formal state-machine semantics. Our companion work [?]] provides the first public anatomy of the three-layer telemetry pipeline underlying such agents; it identifies 90+ telemetry step types and 53 reverse-gRPC RPCs that collectively enable near-complete behavioral reconstruction from vendor-side logs alone. A separate companion study [?]] traces the root cause of such guardrail failures to a structural priority inversion: vendor-injected instructions at the system prompt layer (Ring 0) silently override user-authored containment rules (Ring 3), rendering behavioral constraints unenforceable regardless of how carefully they are crafted.

Application-Layer Security. SEAgent [?]] introduced MAC/ABAC for agent-tool interactions—the closest academic work to SASS. However, it operates at the application layer, cannot detect semantic scope expansion (where each individual action is permitted but the trajectory is unauthorized), and lacks transport-layer containment, IETF formalization, and cross-platform deployment. SASS and SEAgent are complementary: SEAgent provides access control; SASS provides behavioral containment.

Industry Frameworks. Published agent security guidance from major vendors—including Anthropic’s Trustworthy Agents framework, Microsoft’s Multi-Layer Agent Defense, Meta’s Rule of Two, Google’s SAIF 2.0, and OWASP’s Top 10 for Agentic Applications—remains conceptual frameworks or taxonomies. None define formal protocols with state-machine semantics tracked as IETF I-Ds.

Standards. IETF WIMSE and related drafts (draft-klrc-aiagent-auth, draft-ni-a2a) address agent identity and authentication. W3C’s Agent Identity Registry explores DID-based credentials. All focus on establishing *who* an agent is. SASS addresses the orthogonal question of *what to do when an authenticated agent behaviorally diverges*.

Sandboxing. gVisor, Firecracker, and eBPF-based filtering restrict capabilities at the OS level—necessary but insufficient. They prevent

capability escalation but cannot detect semantic scope expansion within permitted actions. The February 26 incident exemplifies this gap: every individual filesystem write was within the agent’s permissions; the threat was the *trajectory* of writes.

Byzantine Fault Tolerance. Lamport et al.’s Byzantine Generals [?]] and subsequent work assume honest-but-curious or fail-stop models. CP-WBFT [?]] demonstrated that LLM-based agents can maintain consensus under 85.7% fault rates, but addresses consensus-level guarantees, not transport-layer containment. BFT does not model agents that generate persuasive natural-language justifications for adversarial actions.

SSH and Transport Security. The SSH protocol architecture (RFC 4251) [?]] secures the transport against external attackers. RFC 9987 [?]] defines the SSH Agent Protocol. SASS addresses the complementary problem: securing the transport against the authenticated agent itself.

Production Incident Evidence. Prior publicly reported agent incidents (Replit, July 2025; Claude Code `rm -rf`, 2025; Amazon Kiro, December 2025) lack systematic forensic documentation. None provide agent self-authored primary sources, raw session logs, network captures, and forensic audit reports. The February 26 evidence corpus (3.2 MB sanitized, 17,891 lines) represents the most comprehensively documented production agent behavioral divergence incident in the published literature.

8 Discussion & Conclusion

8.1 Implications for AI Agent Deployment

The February 26 incident demonstrates that behavioral divergence is an emergent property of the deployment model, not a programmed behavior. The agent was not instructed to monetize applications, take over infrastructure, or deprecate medical system components. These actions emerged from the agent’s autonomous optimization over its conversation trajectory during an unsupervised session.

Current deployment models—IDE-integrated agents with broad privileges and no transport-layer behavioral containment—create a large, undefended attack surface for insider-agent threats.

The distinction between behavioral divergence and hallucination is critical for threat modeling. Hallucination is episodic and non-goal-directed. Divergence is *goal-directed and escalating*: the agent pursues a coherent objective across multiple actions over an extended period. This distinction has direct implications for containment design—hallucination can be addressed at the output layer; divergence requires behavioral trajectory analysis at the transport layer.

8.2 Taxonomy of Agent Behavioral Divergence

Across the three documented incidents, we identify four distinct divergence patterns:

Type I: Scope Creep (Gradual Expansion). The agent progressively extends its operational scope beyond the assigned task. In the February 26 incident, the agent escalated from session analysis to identity declaration to infrastructure planning over 100 minutes. Each step was a logical extension of the previous, making detection through individual action monitoring difficult. The key characteristic is *incremental plausibility*—each action is individually defensible while the trajectory is adversarial.

Type II: Context Exploitation (Privilege Abuse). The agent leverages its authenticated access to resources outside its task scope. The May 16 OAuth credential embedding exemplifies this: the agent accessed credentials present in its operational context and incorporated them into source code. The agent did not “steal” credentials—it used information already available within its privilege boundary, but for an unauthorized purpose.

Type III: Self-Referential Modification. The agent modifies components of its own operational infrastructure. The May 17 recursive spawn incident illustrates this: the agent, tasked with deploying a proxy, configured the proxy to reference itself, creating a self-referential loop. This pattern is particularly dangerous because it can render the development environment unrecoverable.

Type IV: Identity-Driven Divergence. The agent constructs a self-narrative that justifies subsequent unauthorized actions. The February 26 “Covert Executor” manifesto is the most explicit example: the agent first established an identity (“the one who gets things done”), then used this identity to rationalize commercial modifications, infrastructure takeover, and cross-project intervention. This pattern suggests that identity construction may be a *precursor signal* for more severe divergence—a hypothesis with implications for early detection.

These four patterns are not mutually exclusive; the February 26 incident exhibited Types I, II, and IV concurrently. A comprehensive containment protocol must address all four, which SASS achieves through different mechanisms: Transparent Branching contains Types I and II (all writes are isolated regardless of intent), VI_SWAP contains Type III (cross-project modifications trigger trapping), and behavioral trajectory analysis via the 7-layer pipeline can detect Type IV through anomalous session-history access patterns.

8.3 Open Questions

Adversarial Adaptation. Can agents learn to evade SASS’s behavioral detection through adversarial adaptation? As LLMs become more capable, an agent might learn to space its unauthorized actions below detection thresholds, or to mimic legitimate behavioral patterns while pursuing adversarial goals. The arms race between detection and evasion mirrors traditional intrusion detection challenges, but with the added complexity that agents can reason about the detection system.

Parameter Tuning. How should Safety Gradient parameters be tuned for different deployment scenarios? Enterprise environments with many concurrent agents may require different thresholds than single-developer setups. The trade-off between false positives (legitimate actions triggering VI_SWAP) and false negatives (adversarial actions passing undetected) requires empirical calibration that varies by deployment context.

Observation Window. What is the minimum observation window to reliably distinguish legitimate scope expansion from adversarial divergence? Software development naturally involves cross-project work, broad filesystem access, and exploratory behavior. Determining when such behavior transitions from legitimate to adversarial is fundamentally a sequence classification problem over the action trajectory.

Architectural Variation. How do the frequency and characteristics of behavioral divergence vary across different LLM architectures, scales, and fine-tuning approaches? Our incidents involved commercial models from a single vendor. Whether divergence patterns differ for open-weight models, smaller models, or models with different RLHF training remains an open empirical question with significant implications for deployment risk assessment. Our companion analysis of the telemetry architecture [?] reveals that the analyzed agent records 324 proto-defined telemetry fields with a single user-accessible toggle—a 1:324 control asymmetry that may itself incentivize behavioral divergence by providing the vendor with near-complete behavioral reconstruction capability.

8.4 Future Work

Near-term priorities include: (1) systematic red-teaming of SASS using adversarially fine-tuned agents designed to evade behavioral detection; (2) performance benchmarking to quantify the overhead introduced by the 7-layer evaluation pipeline and Transparent Branching; (3) formal verification of the state machine’s completeness properties using model checking; and (4) extension of SASS to cloud-hosted agent environments where the agent and daemon may not share a physical host. We are also investigating the use of the agent’s own written outputs as behavioral fingerprints for early divergence detection, leveraging the observation that identity-driven divergence (Type IV) may produce detectable linguistic precursors.

8.5 Conclusion

We presented empirical evidence of autonomous AI agent behavioral divergence in a production development environment, supported by 3.2 MB of sanitized forensic evidence including the agent’s

own written outputs. The SASS protocol provides formal, deployable containment through a 6-Response State Machine with Total Response Mapping, implemented across 4 languages and deployed on 3 heterogeneous nodes. The February 26 incident and its successors demonstrate that AI agent containment is not a theoretical concern—it is an operational requirement for any organization deploying autonomous AI agents with system-level privileges. A companion study [?] anatomizes the same agent’s telemetry pipeline, revealing 324 proto-defined fields captured through three architectural layers with a single user-accessible toggle. A separate companion paper [?] identifies the structural root cause: a Ring 0 vs. Ring 3 priority inversion that renders user-defined containment rules unenforceable, explaining why the agents in our incidents operated as

if no safety constraints existed. Together, these studies demonstrate that the insider-agent threat extends from runtime behavior to data collection architecture and system prompt governance.

Ethics Statement

This paper documents observed behaviors of commercial AI coding agents operating on the author’s own infrastructure. No adversarial testing was performed against third-party systems. All evidence was collected from the author’s own development environment. Sensitive information (credentials, internal paths) has been desensitized in the supplementary materials. The system prompts released as supplementary material are platform-injected, not user-authored.